

ECE598: Generative Modeling with Diffusion and Flow Matching Models — Homework 2

Prof. Zhizhen Jane Zhao

TA: Yichi Zhang

Instructions

- Homework is due **Monday, Dec. 01, 11:59PM CDT**.
- Everyone must submit individually at Gradescope under **hw2**.
- We reserve the right to reduce the auto-graded score for **hw2** if we detect funny business (e.g., your solution lacks any algorithm and hard-codes answers you obtained from someone else, or simply via trial-and-error with the autograder).
- When submitting to **hw2**, only upload **hw2.flow.py** and **hw2.mdm.py**. Additional files will be ignored.

Task Overview

In this homework, you will implement the flow matching model and the masked diffusion model. We provide the datasets, model architecture, training loop, evaluation metrics, and plotting code. Your task is to:

- Implement a **continuous-time flow matching** model on 1D and 2D toy datasets.
- Compare the flow matching model against the DDPM/DDIM from Homework1.
- Implement a **masked diffusion model (MDM)** on a simple discrete toy dataset.

Datasets

Continuous datasets (Flow Matching). We reuse the continuous toy datasets from Homework 1:

1. **1D 2-Gaussian mixture** (two modes).
2. **2D 8-Gaussian mixture** (points arranged in a ring).
3. **2D S-curve** (nonlinear manifold).

All datasets are already normalized and scaled.

Discrete dataset (MDM). For the masked diffusion model, we use a simple discrete toy dataset:

- Vocabulary $\{0, \dots, V-1, V\}$ with mask token V . $V = 10$.
- Sequence length: $L = 4$.
- Dataset:

$$\text{data} = \{(k, k+1, k+2, k+3 \bmod V)\}_{k=0}^{V-1},$$

e.g., $[0, 1, 2, 3]$, $[3, 4, 5, 6]$, $[8, 9, 0, 1]$, etc.

Model

Continuous data (Flow Matching). The model is the same MLP with time embedding used in Homework 1. You should interpret its output as a velocity field

$$v_\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x, t) \mapsto v_\theta(x, t).$$

Discrete data (MDM). For the discrete dataset, we provide a simple sequence model (e.g., Transformer/MLP) that outputs logits over the vocabulary at each position:

$$p_\theta(x_0^i \mid x_t) \in \Delta^{V-1}, \quad i = 1, \dots, L.$$

You should interpret its output as conditional distributions over the original (unmasked) tokens given the corrupted sequence x_t .

You do **not** need to modify any model definitions or the training loop outside of the specified methods.

Training

Training scripts are provided. You only need to implement the missing methods so that training runs correctly. Hyperparameters (learning rates, batch sizes, etc.) are chosen for you and should already work without tuning.

Evaluation

Continuous data (Flow Matching). For the 1D and 2D toy datasets, we use exactly the same metrics as in Homework1:

- For **1D datasets**, we compute the **1-Wasserstein distance**.
- For **2D datasets**, we compute the **sliced Wasserstein-2 distance**.

These metrics allow direct comparison between Flow Matching and the DDPM/DDIM results from Homework1.

Discrete data (Masked Diffusion Model). For the discrete toy dataset (vocabulary $\{0, \dots, V-1, V\}$ with mask token V), we evaluate generation quality using two metrics:

- **KL divergence:** We compute the Kullback–Leibler divergence between the empirical distribution of generated sequences and the ground truth distribution of sequences in the dataset.
- **Validity accuracy:** A generated sequence is considered *valid* if it belongs to the ground truth set. The *accuracy* metric is the proportion of generated samples that fall into this set.

These two metrics measure diversity and correctness. All evaluation code is provided and will automatically run after training and sampling.

Visualization

The plotting code from Homework1 is reused. It will automatically generate:

- Distribution plots comparing real data and generated samples.
- Sample path plots (trajectories $x_T \rightarrow x_0$).

This will help you visualize the flow matching sampling.

Your Implementation Task

Part I: Flow Matching (Continuous Data)

You will complete the following methods in the `FlowMatching` class:

1. `loss(x0, x1, t)` Implement the flow matching training loss:

- Construct the interpolated points

$$x_t = (1 - t) x_0 + t x_1,$$

where x_0 are data samples and x_1 are noise samples.

- The target velocity is

$$v_{\text{target}} = x_1 - x_0.$$

- Compute the loss

$$L_{\text{FM}}(\theta) = \mathbb{E}_{t, x_0, x_1} \|v_\theta(x_t, t) - (x_1 - x_0)\|^2.$$

2. `sample(n, steps)` Implement sampling by Euler integration of the ODE

$$\frac{dx}{dt} = v_\theta(x, t)$$

from $t = 1$ to $t = 0$:

- Initialize x at $t = 1$ as

$$x^{(0)} \sim \mathcal{N}(0, I).$$

- Define a uniform time grid

$$t_k = \frac{k}{\text{steps}}, \quad k = 1, \dots, \text{steps},$$

with step size $\Delta t = 1/(\text{steps})$.

- Iterate backwards over k from `steps` to 1 and update

$$x \leftarrow x - \Delta t \cdot v_\theta(x, t_k),$$

corresponding to integrating from $t = 1$ to $t = 0$.

- Return the final x at $t = 0$ together with the full trajectory $\{x_t\}$.

Part II: Masked Diffusion Model (MDM, Discrete Data)

You will complete the following methods in the `MDM` class. Recall that data tokens lie in $\{0, \dots, V - 1\}$ and the mask token has index V .

1. `qt_sample(x0, t)` Implement the forward corruption process $q(x_t \mid x_0, t)$:

- For each sequence position i and each example in the batch, independently mask with probability t :

$$q(x_t^i = V \mid x_0^i, t) = t, \quad q(x_t^i = x_0^i \mid x_0^i, t) = 1 - t.$$

- Return:

- a boolean mask indicating positions where $x_t^i = V$,
- the corrupted sequence x_t with masked positions set to V .

2. `loss(x0, t)` Implement the LLaDA-style training loss:

- Sample x_t from $q(x_t \mid x_0, t)$ using `qt_sample`.

- Let $p_\theta(x_0^i | x_t)$ be the model's predicted distribution over original tokens at position i .
- Define the loss

$$L_{\text{MDM}}(\theta) = -\mathbb{E}_{t, x_0, x_t} \left[\frac{1}{t} \sum_{i=1}^L \mathbf{1}\{x_t^i = V\} \log p_\theta(x_0^i | x_t) \right].$$

- In practice, you should:
 - compute the log-probability of the true token x_0^i at each position,
 - mask out unmasked positions ($x_t^i \neq V$),
 - weight by $1/t$ per example,
 - average over the batch to obtain a scalar loss.

3. `sample(n, T, temperature)` Implement iterative sampling over T masked-diffusion steps:

- Initialize $x^{(0)}$ as a fully masked batch:

$$x_{b,i}^{(0)} = V \quad \text{for all } b, i.$$

- Define a time schedule

$$t_0 = 1, \quad t_1, \dots, t_T = 0$$

(e.g., linearly spaced).

- For each step $k = 0, \dots, T - 1$:
 - Evaluate the model at $(x^{(k)}, t_k)$ to obtain logits over $\{0, \dots, V\}$ at each position.
 - Apply Gumbel noise with temperature scaling to the logits, convert to probabilities, and sample candidate tokens in $\{0, \dots, V - 1\}$.
 - Use the provided schedule `get_num_transfer_tokens` to decide how many positions with value V to convert in each sequence.
 - For each sequence, select that many masked positions and replace V by the sampled tokens.
 - This yields $x^{(k+1)}$.
- After T steps, return the final sequences $x^{(T)}$ and the full trajectory $\{x^{(k)}\}_{k=0}^T$.

Deliverables

- Submit your completed `hw2_flow.py` and `hw2_mdm.py` files to Gradescope under `hw2`.

Exploration

After finishing the required implementation, you are encouraged to:

- Compare the qualitative sample paths of flow matching vs. DDPM and DDIM on the same datasets.
- Vary the number of ODE integration steps `steps` in flow matching and observe how it affects sample quality and convergence.
- Experiment with different unmasking schedules (i.e., different T and time grids) and temperatures in MDM sampling.

Note

As in Homework 1, the defaults are chosen so that you should obtain reasonable performance without tuning. However, feel free to play with parameters.